Baseband and Cryptographic Subsystem

RTL Architecture and Verification Report

Project 4 - Group 2

Spandan Bharadwaj (230102108)
Patel Dhruv Utkarsh (230102112)
Shreyas Sagar (230102118)
Akhilesh Arun Reddy (230108003)

September 29, 2026

Contents

1 Project File Structure **1**

2 Assumptions and Memory Map **3**

3 Top-Level Architectural Specifications **4**
3.1 1. APB Wrapper (`apb_wrapper.v`) 4
3.2 2. Loopback Multiplexer (`loopback_mux.v`) 5
3.3 3. Digital Baseband (DBB) 5
3.4 4. PRESENT-80 Crypto Engine 5

4 Verification Report and Test Plan **5**
4.1 Testbench Execution Summary 5
4.2 Simulation Execution Instructions 6
4.3 RTL Modifications Implemented During Verification 6

5 Elaboration and Compilation Progress **7**

1 Project File Structure

The project submission consists of a well-organized root directory containing the hardware description folders, verification suites, and system documentation.

1. RTL Folder (Hardware Description)

This folder contains the core synthesizable Verilog modules that define the system's physical logic.

- `baseband_crypto_macro.v`**:** This is the top-level macro that bridges the DBB front-end with the PRESENT-80 crypto back-end in a single digital footprint.
- `apb_wrapper.v`**:** Serves as the memory-mapped PCLK-domain slave interface for the CPU. It enforces the system's strict "Streaming-Only" constraint by actively stalling the APB bus using `PREADY` during cryptographic operations.
- `loopback_mux.v`**:** The central routing crossbar. It directs data flows between Live RF Mode, DBB Loopback Mode, and Crypto Loopback Mode, completely isolating test traffic from physical analog pins.
- `dbb_top.v` **& Submodules:** Contains the full 5-stage digital baseband pipeline (Synchronizer, Oversampler, Voter, Manchester FSM, and Framer) designed to filter noise and decode RF signals.
- `present_top.v` **& Submodules:** The 31-round PRESENT-80 cryptographic engine utilizing parallel S-boxes and a zero-cost wire-routed permutation layer.

2. TB Folder (Testbenches)

Contains the complete suite of SystemVerilog testbenches used to rigorously verify the RTL against corner cases and asynchronous timing issues.

- `Golden_Vectors.sv`**:** Proves mathematical correctness of the cipher against OpenCores standards.
- `Preamble_Validation_TB.sv`**:** Validates the byte framer's strict requirement for the `8'b10101011` preamble, ensuring corrupted headers correctly trigger error flags rather than outputting garbage data.
- `Mode_0.sv`**:** Validates autonomous system flow by driving a clock-aligned synthetic wave into the RF pin.
- `Mode_0_async_data.v`**:** Proves the system's resilience against real-world, unaligned analog RF signal transitions by phase-shifting the input.
- `Mode_1.sv`**:** Injects synthetic physical waveforms directly into the baseband via the APB bus, proving DBB voter and Manchester FSM logic.
- `Mode_switching.sv`**:** Toggles between modes to validate that state retention bleeding does not occur.
- `Corner_Cases_TB.sv`**:** Advanced verification suite targeting read-on-assert race conditions, dynamic RF timing jitter, and carrier frequency drifts.
- `Reset_stress.sv` & `Reset_Stress_final.sv`**:** Injects hostile random resets midstream to prove the hardware gracefully recovers without deadlocking.

3. Documentation Files

- `Verification_Report.docx`: Details the execution, methodology, and architectural fixes applied during testing.
- `baseband_crypto_macro_elab_check.log.txt`: The official Synopsys Design Vision output proving a flawless compilation and 0-error elaboration.
- `test_plan_v2.txt`: The Cadence Xcelium simulation logs capturing terminal outputs for all passed verification suites.

2 Assumptions and Memory Map

Assumptions:

1. Based on our understanding of the TI Paper, we are proposing the use of **PREAMBLE**, with its value being **8'b10101011**.
   Logic: Once the PREAMBLE has been verified, the unit will start accepting valid data. This has been implemented in both Mode 0 and Mode 1.

2. Oversampling Ratio is 6 and we need a register size of 432.
   *Logic: In Mode 1, we are assuming that the incoming data sent by the master is raw oversampled bits, i.e., the input has already been through the process of oversampling. Hence, this input goes directly into Majority Voter. When master writes to the slave, the current spec writes the data in two cycles (ADDR_DATA_0 and ADDR_DATA_1). As the OSR is 6, we will be looking at 6 bits in order to infer 1 bit (as per Manchester Coding). In order to pack to 64 decoded bits, the register inside our unit should be of size 432 because 64*OSR + (PREAMBLE Length)*OSR (64*6 + 8*6 = 432).*

The updated Memory Map is as follows:

tableDark			
0x00	CTRL_REG	R/W	Bits [1:0]: Mode (0=Live, 1=DBB Loopback, 2=Crypto Loopback)
0x04	STATUS_REG	R/O	Bit 0: DONE (Crypto), Bit 1: DATA_READY (DBB)
0x08	ADDR_KEY_0	W/O	Lower 32 bits of PRESENT-80 Key
0x0C	ADDR_KEY_1	W/O	Middle 32 bits of PRESENT-80 Key
0x10	ADDR_KEY_2	W/O	Upper 16 bits of PRESENT-80 Key
0x14	ADDR_DATA_0	W/O	Bits [31:0] of Plaintext / Raw DBB Oversample stream
0x18	ADDR_DATA_1	W/O	Bits of Plaintext/DBB (Writing this triggers Mode 2 FSM)
0x1C - 0x44	ADDR_DATA_2 to ADDR_DATA12	W/O	Bits of Raw DBB Oversample stream
0x48	ADDR_DATA_13	W/O	Upper 16 bits [431:416] of DBB stream (Writing this triggers Mode 1 FSM)
0x4C	ADDR_DATA_OUT_0	R/O	Lower 32 bits of Ciphertext / Decoded DBB bytes
0x50	ADDR_DATA_OUT_1	R/O	Upper 32 bits of Ciphertext / Decoded DBB bytes

Enteries in *Green* are newly added, while those in *Red* have a changed address value.

3 Top-Level Architectural Specifications

The system operates as a linear pipeline: Async RF input → DBB front end → Loopback routing → PRESENT-80 crypto → APB slave → CPU.

3.1 1. APB Wrapper (`apb_wrapper.v`)

Operates in the `PCLK` / `PRESETn` clock domain. Acts as the slave interface to the system CPU, decoding addresses to update control/key registers.

- **Streaming Constraint:** Enforces "Streaming-Only" rules by strictly pulling `PREADY` low to freeze the APB bus during the 32-cycle crypto operation.

- **Bus Gating:** `PSEL` acts as a strict clock-enable, freezing internal peripheral flip-flops whenever the CPU addresses other devices on the bus.

3.2 2. Loopback Multiplexer (`loopback_mux.v`)

Operates in the `clk` / `rst_n` domain. Controlled by `ctrl_mode[1:0]`, it routes data to isolate and unit-test internal blocks without requiring live analog RF signals.

- **Mode 0 (Live):** Routes `rfid_rx_bit` → DBB → Crypto.
- **Mode 1 (DBB Loopback):** APB synthetic bits → Serializer → DBB → APB Read. Employs operand isolation to clamp the crypto inputs to zero, saving dynamic power.
- **Mode 2 (Crypto Loopback):** APB 64-bit Plaintext → Crypto → APB Read (bypassing DBB entirely).

3.3 3. Digital Baseband (DBB)

A five-stage sequential pipeline designed to filter high-frequency noise and frame bytes.

- `dbb_sync`**:** 2-stage D-Flip-Flop synchronizer safely pulls the asynchronous RF bit into the clock domain to prevent metastability.
- `dbb_oversampler` **&** `dbb_voter`**:** Samples the signal to create a sliding historical window. The voter is a purely combinational adder-tree filtering noise via majority logic.
- `dbb_manchester_fsm`**:** A 3-state FSM (IDLE, SKIP, ARMED) that isolates center "data edges" while blinding itself to boundary edges.
- `dbb_framer`**:** Groups the extracted bits into 64-bit words, hunting specifically for the preamble.

3.4 4. PRESENT-80 Crypto Engine

An ultra-lightweight Substitution-Permutation Network operating on a 64-bit plaintext with an 80-bit key over 31 rounds.

- **Hardware Optimization:** The permutation layer (`pLayer`) is achieved via 0-cost explicit wire routing, preventing synthesis tools from inferring arithmetic mod/div logic.

4 Verification Report and Test Plan

The RTL underwent aggressive simulation using Cadence Xcelium. All testbenches passed successfully, confirming mathematical accuracy, timing robustness, hardware recovery, and strict preamble enforcement.

4.1 Testbench Execution Summary

1. **Mode 2 Golden Vectors (`Golden_Vectors.sv`):** Bypasses the DBB to validate the mathematical correctness of the PRESENT-80 crypto engine using OpenCores golden vectors. Explicitly verified the 32-cycle `PREADY` APB stall mechanism.
2. **Preamble Validation & Error Recovery (`Preamble_Validation_TB.sv`):** Evaluates the system's strict enforcement of the `8'b10101011` preamble in Mode 0 and Mode 1. Validates that the payload is exclusively fetched if the preamble is perfect. The testbench proves:

- **Positive Preamble Locking:** A valid preamble correctly asserts `STATUS_PREAMBLE_LOCKED` (Status[2]) and `STATUS_DBB_READY` (Status[1]), successfully fetching the payload.
- **Corrupted All-Zero Preamble Rejection:** An all-zero header correctly triggers `STATUS_MODE1_ERR` (Status[3]) and halts data fetching.
- **Delimiter Symbol Mutation:** Mutating the preamble delimiter from `8'b10101011` to `8'b10101010` triggers an error timeout (Status[3]) and forces the framer to reject the packet.
- **Autonomous Pipeline Recovery:** Proves the pipeline can autonomously flush a rejected preamble and immediately lock onto a subsequent valid frame without requiring a manual hardware reset.

3. **Mode 0 Synchronous Execution (`Mode_0.sv`):** The testbench acts as a physical antenna, driving the `rfid_rx_bit` pin with a 432-bit synthetic oversampled radio wave perfectly aligned to the clock edge.

4. **Mode 0 Asynchronous Execution (`Mode_0_async_data.v`):** Stress-tests the `dbb_sync` synchronizer by phase-shifting the incoming radio wave by a sub-cycle fractional delay (3.7ns). Proved resilience against real-world, unaligned analog signals.

5. **Mode 1 Loopback (`Mode_1.sv`):** Bypasses the physical antenna to inject synthetic physical waveforms directly into the baseband via a 432-cycle APB stall, proving DBB voter and Manchester FSM logic.

6. **Mode Switching Permutations (`Mode_switching.sv`):** Aggressively toggles between Modes 0, 1, and 2 using alternating data payloads. Proves the pipeline soft-reset logic prevents stale state retention and data bleeding.

7. **Random Reset Stress (`Reset_Stress_final.sv`):** Uses parallel SystemVerilog threads to violently inject hostile resets (`PRESETn = 0`) at random intervals mid-stream (e.g., during an APB stall). Proved the hardware gracefully flushes and recovers without deadlocking.

4.2 Simulation Execution Instructions

To execute the Preamble Validation testbench strictly in the terminal without launching the Cadence graphical interface, execute the following command in your root project directory:

```
[EE5239G2@avanti ~]$ xrun -access +rwc -sv -top Preamble_Validation_TB
RFID_v2/TB/Preamble_Validation_TB.sv RFID_v2/RTL/*.v
```

Note: The above command utilizes the Xcelium single-step compilation and simulation engine exactly as mapped in the provided test plan.

4.3 RTL Modifications Implemented During Verification

Based on testing failures, the following architectural fixes were applied to the RTL:

- **Single-Cycle Triggers:** Added `&& PREADY` logic to APB triggers (e.g., `apb_trigger_m2`). This prevented rogue secondary executions when the CPU held `PENABLE` high during stalls.
- **Data Race Prevention:** Introduced a 1-cycle delay (`m2_trigger_delayed`) in `loopback_mux.v` to ensure the `crypto_start` pulse fires only after the APB payload safely latches.

- **Universal Soft Reset:** The DBB pipeline reset (`dbb_soft_rst_n`) was tied to a `ctrl_mode_changed` detector to violently flush pipelines upon mode switches, preventing stale retention.
- **Stale Flag Wiping:** The `STATUS_REG` was modified to wipe completely on any `CTRL_REG` modification, eliminating ghost status reads.

5 Elaboration and Compilation Progress

The full Verilog architecture was compiled and elaborated using Synopsys `design_vision` (Version T-2022.03-SP5).

- The tool successfully built all hierarchical modules starting from `baseband_crypto_macro` down to `sbox` and `apb_wrapper`.
- Inferred hardware matches the specification exactly, with all intended Flip-Flops accurately synthesized (e.g., 432-bit `apb_test_data`, 80-bit `crypto_key`).
- The `check_design` pass confirmed **0 errors**. A few standard LINT-29 and LINT-1 warnings were flagged regarding deliberate wire pass-throughs (e.g., inside `perm_layer.v`), which are architecturally correct for 0-cost routing constraints.